

SMC SystemC/TLM-2.0 IP Library — Verification Plan

Contents

- SMC SystemC/TLM-2.0 IP Library — Verification Plan
 - A. Verification Infrastructure
 - A.1 Frameworks & tools
 - A.2 Common test bench skeleton
 - A.3 Common test helpers (provided as `smc_test_utils.h`)
 - A.4 Coverage goals
 - A.5 Pass / fail criteria
 - A.6 Regression strategy
 - B. Per-IP Unit Test Plans
 - B.1 SMC Fabric
 - B.2 AXI Filter
 - B.3 CPU Cluster (Rocket wrapper / stub)
 - B.4 PLIC
 - B.5 CLINT
 - B.6 WDT
 - B.7 Bus Error Unit
 - B.8 Boot ROM
 - B.9 Scratchpad SRAM
 - B.10 Reset Unit
 - B.11 PLL Wrapper
 - B.12 MISC Wrapper
 - B.13 Debug Module
 - B.14 DMA Engine
 - B.15 Memory Zeroer
 - B.16 Mailbox Unit
 - B.17 System Timer OCTS
 - B.18 eFuse
 - B.19 UART 16550
 - B.20 Log Engine
 - B.21 I2C Controller
 - B.22 AVSBus Controller
 - B.23 GPIO
 - B.24 Telemetry Receiver
 - B.25 PVT Wrapper

- C. SoC-Level Integration Tests (smc_top)
 - C.1 Boot sequence (TC-SOC-BOOT-NNN)
 - C.2 FLR sequence (TC-SOC-FLR-NNN)
 - C.3 Interrupt aggregation (TC-SOC-IRQ-NNN)
 - C.4 DMA + fabric scenarios (TC-SOC-DMA-NNN)
 - C.5 OCTS multi-chiplet (TC-SOC-OCTS-NNN)
 - C.6 eFuse + Security (TC-SOC-SEC-NNN)
 - C.7 Logging (TC-SOC-LOG-NNN)
 - C.8 Telemetry (TC-SOC-TLM-NNN)
 - C.9 Stress / Random (TC-SOC-STR-NNN)
- D. Functional Coverage Plan
- E. Test Deliverables
- F. Open Items / Out of Scope

SMC SystemC/TLM-2.0 IP Library — Verification Plan

This document defines the verification plan for the SystemC/TLM-2.0 (Loosely-Timed) models specified in `02_SMC_IP_LowLevel_Design.md`. It targets two levels of verification:

- **L1 — Unit tests:** each IP exercised in isolation in a minimal test-bench.
- **L2 — Integration / SoC-level tests:** full `smc_top` exercised by the SMC firmware ROM image (or scripted CPU stub) under representative use cases.

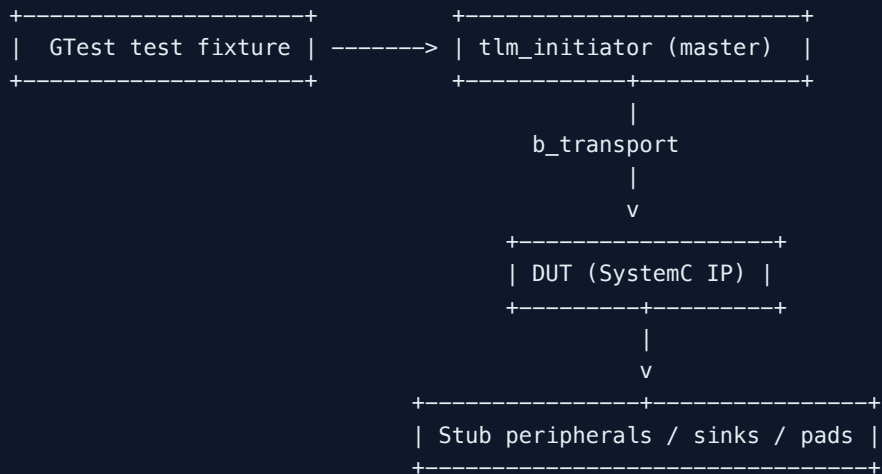
A consistent infrastructure (Section A) is used across all tests.

A. Verification Infrastructure

A.1 Frameworks & tools

Tool	Purpose
SystemC 2.3.4+ / TLM-2.0	Simulation kernel
GoogleTest	Test runner, assertions
CMake / CTest	Build & orchestration
Icov + gcovr	C++ line/branch coverage
Python <code>pytest</code> + <code>scapy/serial</code>	End-to-end host-side stimulus generation (UART, I2C scripts)
VCD trace + GTKWave	Wave debug for IRQ/strap signals
Spike / RV ISS or scripted CPU stub	Drives firmware-style register accesses

A.2 Common test bench skeleton



A.3 Common test helpers (provided as `smc_test_utils.h`)

- `smc_master::write32(addr, val, prot=DATA_SECURE_PRIV)`
- `smc_master::read32(addr, prot=DATA_SECURE_PRIV)`
- `smc_master::write64(addr, val)` / `read64(addr)`
- `expect_irq(sc_in<bool>& irq, sc_time within)`
- `wait_until_clear(reg_addr, mask, timeout)`
- `inject_atb_stream(...)` — for telemetry tests
- `gpio_force_pad(pin, value)` — for strap tests

A.4 Coverage goals

Metric	Goal
Line coverage per IP	≥ 95%
Branch coverage per IP	≥ 90%
Register coverage (every register read & written)	100%
Reset value check (per register)	100%
IRQ source coverage (every interrupt at least once)	100%
Functional coverage (per scenario list below)	100%

A.5 Pass / fail criteria

A test passes only if all of:

- All `EXPECT_*` / `ASSERT_*` succeed.
- No SystemC warnings/errors logged.

- No timeouts (`SC_REPORT_FATAL` from watchdog inside test).
- Coverage data merged into the global report.

A.6 Regression strategy

- **Tier 0 (smoke):** <5 min, every commit. Reset, register-default, basic R/W per IP.
 - **Tier 1 (unit):** <30 min, nightly. Full IP unit-test suite.
 - **Tier 2 (integration):** <2 h, weekly. `smc_top` boot, FLR, telemetry, DMA stress.
 - **Tier 3 (long-soak):** 8 h, weekly. Random stimulus, error injection, long-running OCTS sync drift.
-

B. Per-IP Unit Test Plans

The following sections define test cases per IP. Each test has an ID `TC-<IP>-NNN`.

B.1 SMC Fabric

ID	Title	Pre-conditions	Stimulus	Expected	Coverage
TC-FAB-001	Default local alias decode	After reset; LOCAL_BASE=0xC000_0000	Write/read at 0xC003_8000	Routes to DMA control socket	Local decode
TC-FAB-002	Reprogram GLOBAL_BASE	LOCAL_BASE != GLOBAL_BASE	Set GLOBAL_BASE=0x4000_0000; access at GLOBAL+offset	Routes correctly	Global decode
TC-FAB-003	Alias remap region	Configure ALIAS_REMAP[0] {base,size,offset,cacheable=1}	Issue access in remap range	Address translated; cacheable bit set in extension	Alias remap
TC-FAB-004	M-mode remap	Configure MMODE_REMAP[0]; set source prot[0]=1	Access shared addr	source_id = MMODE_ID, address translated	Privilege remap
TC-FAB-005	Xvisor remap	Same as above with prot[0]=0	—	source_id = OTHER_ID	Privilege remap
TC-FAB-006	Out-of-range → output_axi	Address > REGION_SIZE	Issue write	Forwarded to output_axi	Outbound path
TC-FAB-007	Decode error	Issue access to unmapped local offset	b_transport returns DECERR	Error propagation	
TC-FAB-008	Multi-master ordering (LT)	Two initiators write to same target	Order respected per quantum	Functional correctness	

B.2 AXI Filter

ID	Title	Stimulus	Expected
TC-FLT-001	Allow secure	<code>allow_ns=0</code> , prot[1]=0	Pass through
TC-FLT-002	Block non-secure	<code>allow_ns=0</code> , prot[1]=1	DECERR; data 0xBADCAB1E
TC-FLT-003	Source ID match	SRCID match	Pass
TC-FLT-004	Source ID mismatch	SRCID mismatch	Block
TC-FLT-005	Address range	Addr inside / outside <code>BASE..BASE+MASK</code>	Pass / Block
TC-FLT-006	AXI-Lite full prot match	prot==awprot_req	Pass
TC-FLT-007	AXI-Lite full prot mismatch	prot!=awprot_req	DECERR
TC-FLT-008	16-instance independence	Configure 16 different filters	Each acts independently

B.3 CPU Cluster (Rocket wrapper / stub)

ID	Title	Stimulus	Expected
TC-CPU-001	Reset vector	Set RESET_VECTOR_0=0xC0040000; release rst	First fetch from 0xC0040000
TC-CPU-002	Per-core enable	Enable cores 0..3 individually	Only enabled cores fetch
TC-CPU-003	LOCAL/GLOBAL_BASE programming	Write GLOBAL_BASE; access GLOBAL addr	Reaches local target
TC-CPU-004	INIT_MEM_DONE handshake	Trigger SRAM init	INIT_MEM_DONE asserts
TC-CPU-005	Disable SRAM auto init	Assert disable; reset	INIT_MEM_DONE never asserts
TC-CPU-006	NMI from BEU	Trigger BEU error	CPU receives NMI
TC-CPU-007	Timer IRQ from CLINT	mtimecmp expires	Hart enters trap
TC-CPU-008	Software IRQ from CLINT	Write MSIP[0]	Hart 0 takes IRQ
TC-CPU-009	External IRQ via PLIC	Inject IRQ#283 (temp)	Hart claims, completes
TC-CPU-010	Boot ROM execution (stub mode)	Run scripted boot sequence	Sequence completes

B.4 PLIC

ID	Title	Stimulus	Expected
TC-PLIC-001	Reset values	Read all priority/enable/threshold	All 0
TC-PLIC-002	Single source	Set priority 7, enable, raise IRQ	Context output asserts
TC-PLIC-003	Threshold filter	Threshold=5, source priority=4	No IRQ
TC-PLIC-004	Claim/complete	Two pending sources, claim returns highest	OK
TC-PLIC-005	Multi-context routing	Enable IRQ on context 0 only	Context 1 stays low
TC-PLIC-006	All 332 sources	Walk through every source	Each one delivered
TC-PLIC-007	Mailbox+SEP+UART aggregation	Inject from mailbox & UART simultaneously	Both delivered
TC-PLIC-008	Reserved sources	Inject on reserved IDs	Tied to 0; ignored

B.5 CLINT

ID	Title	Stimulus	Expected
TC-CLI-001	mtime increments	Run for 1ms simulated	mtime advances by <code>1ms / clk_ref_period</code>
TC-CLI-002	mtimecmp triggers MTIP	mtimecmp = mtime+100	MTIP asserts after 100 ticks
TC-CLI-003	MSIP per core	Set MSIP[2]=1	Only msip_out[2] asserts
TC-CLI-004	64-bit atomic read	Read mtime LO/HI	High word stable across LO read

B.6 WDT

ID	Title	Stimulus	Expected
TC-WDT-001	Default disabled	Reset → leave disabled	No timeout
TC-WDT-002	Warning IRQ	Enable, set WARN_LOAD; wait	warning_irq_o asserts
TC-WDT-003	Reset request	Don't kick after warning	reset_req_o asserts
TC-WDT-004	Pet to clear	Kick before timeout	No IRQ
TC-WDT-005	Debug disable	Set dbg_disable; force halted	No timeout fires

B.7 Bus Error Unit

ID	Title	Stimulus	Expected
TC-BEU-001	Decode error capture	Snoop a DECERR transaction	CAUSE.DEC=1; ADDR captured
TC-BEU-002	Slave error capture	Snoop SLVERR	CAUSE.SLV=1
TC-BEU-003	Timeout	Inject pseudo-timeout event	CAUSE.TIMEOUT=1
TC-BEU-004	NMI assertion	Any error	nmi_o asserted until cleared
TC-BEU-005	Mask	Mask DEC	DEC errors do not raise NMI

B.8 Boot ROM

ID	Title	Stimulus	Expected
TC-ROM-001	Image load	<code>load_image("smc_rom.bin")</code>	First word matches file
TC-ROM-002	Endianness flip	flip_endianness_i=1	Bytes within each 64-bit word swapped
TC-ROM-003	DMI access	Request DMI	dmi_data returned, allowed
TC-ROM-004	Write attempt	Write to ROM	DECERR
TC-ROM-005	Out-of-range	Read beyond image size	DECERR

B.9 Scratchpad SRAM

ID	Title	Stimulus	Expected
TC-SPM-001	Auto-init on reset	Cold reset; disable_auto_init=0	All bytes 0; init_done_o=1
TC-SPM-002	Auto-init disabled	disable_auto_init=1	Bytes preserved
TC-SPM-003	DMI R/W	Use DMI	Direct memory access works
TC-SPM-004	64-bit access	write64/read64	Round-trip equal

B.10 Reset Unit

ID	Title	Stimulus	Expected
TC-RST-001	Cold reset propagation	Assert/deassert rst_cold_n	All rst_primary_* assert/deassert
TC-RST-002	Powergood gating	rst released before powergood	Outputs stay low until powergood=1
TC-RST-003	FLR sequence	Assert cfg_flr_pf_active	After PRE_RESET ticks isolation asserts; after RESET ticks reset asserts
TC-RST-004	skip_mem_repair on FLR	FLR active	skip_mem_repair_o=1
TC-RST-005	Software isolate	Write ISOLATE_REQ_REG	Matching pin asserts
TC-RST-006	Pin isolate	Drive isolate_req_pin_i; PINEN=1	Output asserts

B.11 PLL Wrapper

ID	Title	Stimulus	Expected
TC-PLL-001	Reset values	Read PLL_STATUS	Locked=0
TC-PLL-002	Lock after enable	Enable + program div	Locked=1 after <code>lock_delay_ns</code>
TC-PLL-003	Re-lock on reprogram	Change M	Locked=0→1
TC-PLL-004	freq_mhz_o consistency	Program div=10	freq_mhz_o = ref/10

B.12 MISC Wrapper

ID	Title	Stimulus	Expected
TC-MIS-001	Scratch R/W	Write/read all 64 regs	Round-trip OK
TC-MIS-002	NDM reset request	Write NDM_RESET_REQ	PLIC source #267 fires
TC-MIS-003	Read-only IDs	Try to write CHIP_ID	Write ignored

B.13 Debug Module

ID	Title	Stimulus	Expected
TC-DM-001	DMACTIVE handshake	Set dmactive=1	DMSTATUS reflects active
TC-DM-002	Halt request	Set haltreq	halt_req_o asserts
TC-DM-003	Resume	Set resumereq	halt_req_o de-asserts
TC-DM-004	NDMRESET	Set ndmreset	ndmreset_o pulses
TC-DM-005	System Bus Access write	Configure SBADDR, SBDATA	jtag2axi_out generates b_transport
TC-DM-006	SBA read	SBCS read mode	rdata returned via SBDATA

B.14 DMA Engine

ID	Title	Stimulus	Expected
TC-DMA-001	Linear copy	SRC=A, DST=B, LEN=N	B[0..N) == A[0..N)
TC-DMA-002	2D transfer	Configure stride	Tile copied correctly
TC-DMA-003	Repeat count	Configure repeat=4	Pattern copied 4 times
TC-DMA-004	Scatter-gather	Build SG descriptor	Concatenation correct
TC-DMA-005	IRQ on completion	Enable IRQ	irq_o asserts; STATUS.busy=0
TC-DMA-006	Abort mid-transfer	Start; write abort	STATUS.busy=0; partial transfer; no hang
TC-DMA-007	Config lock	CFG_LOCK=1; try to change	Write ignored / DECERR
TC-DMA-008	AXI error propagation	Target returns SLVERR	STATUS.error=1; IRQ
TC-DMA-009	Round-robin (multi-ctrl)	Two ctrl interfaces	Fair distribution
TC-DMA-010	DMI fast path	DST is DMI-able SRAM	Faster transfer; correctness

B.15 Memory Zeroer

ID	Title	Stimulus	Expected
TC-ZER-001	Basic zero	DEST=A, SIZE=4KB	A[0..4KB) == 0
TC-ZER-002	Unaligned start	DEST=A+3	Start aligned correctly; unmodified bytes preserved
TC-ZER-003	Burst boundary	SIZE crosses 4KB page	Multiple bursts; correct
TC-ZER-004	Abort safe	Start large zero; abort	busy_o falls; no hang
TC-ZER-005	IRQ on completion	Enable IRQ	Fires once
TC-ZER-006	Error reporting	Target returns SLVERR	CTRL_STATUS.error=1

B.16 Mailbox Unit

ID	Title	Stimulus	Expected
TC-MB-001	Push/pop single	Write WRITE_DATA; read READ_DATA on inbound	Round-trip equal
TC-MB-002	FIFO depth	Push 8; push 9th	9th sets overflow; ERROR_FLAGS.write_error=1
TC-MB-003	Empty read	Read empty	underflow; previous data returned
TC-MB-004	Write threshold IRQ	Set WIRQT=4; push 5	inbound_irq fires
TC-MB-005	Read threshold IRQ	Set RIRQT=2; have 3	outbound_irq fires
TC-MB-006	IRQ enable mask	Disable IRQEN	irq stays low
TC-MB-007	Flush via CTRL	Push N; flush	STATUS empty
TC-MB-008	Independence between pairs	Use pairs 0 and 5	No cross-talk
TC-MB-009	All 32 pairs	Walk through every pair	Each works

B.17 System Timer OCTS

ID	Title	Stimulus	Expected
TC-OCTS-001	PRIMARY start	is_primary=1; write START	timer_sync_load_o pulses
TC-OCTS-002	PRIMARY credit gen	CREDIT_VAL=10	credit_o pulses every 10 ticks
TC-OCTS-003	SECONDARY sync_load	feed timer_sync_load_i	counter loads preset
TC-OCTS-004	SECONDARY credit accept	feed credit pulses	counter advances
TC-OCTS-005	Credit expired	No credit pulses	CREDIT_EXPIRED counter increments
TC-OCTS-006	Pulse width	PULSE_WIDTH=5	sync_load_o high for 5 ticks
TC-OCTS-007	64-bit count atomicity	Read LO/HI/LO	atomic value reconstructable
TC-OCTS-008	Sync between 1 PRIMARY + 3 SECONDARY	Wire pulses	All counts within ± 1 cycle
TC-OCTS-009	Reset behavior	Assert rst	counter=0; outputs low

B.18 eFuse

ID	Title	Stimulus	Expected
TC-EFU-001	Sense complete	Cold reset	fuse_sense_done_o eventually asserts
TC-EFU-002	Shadow R from SMC	Read shadow	Returns programmed value
TC-EFU-003	Program bit	Write 1 to fuse bit	Sets bit; subsequent read sees 1
TC-EFU-004	Set-once enforcement	Try to clear	Write ignored
TC-EFU-005	Lock	Set HW lock; program	Program rejected
TC-EFU-006	LC_STATE PROD blocks JTAG write	lc_state=PROD; JTAG write	DECERR + 0xBADCAB1E
TC-EFU-007	LC_STATE PROD allows chiplet ID read	Read CHIP_ID via JTAG	OK
TC-EFU-008	RMA token	Write RMA_SOP_TOKEN	unlock SOP MMR
TC-EFU-009	Security disable bypass	security_disable_i=1	Shadow regs accessible immediately
TC-EFU-010	Secure test mode	secure_tm_i=1	Sensitive regs masked appropriately
TC-EFU-011	BIRA repair_data	Read 16-kbit field	Returns provisioned value

B.19 UART 16550

ID	Title	Stimulus	Expected
TC-URT-001	Default registers	After reset	Match 16550 defaults
TC-URT-002	Baud rate cfg	DLAB=1, write DLL/DLM	Effective baud = $\text{clk}/(16*\text{div})$
TC-URT-003	TX char	Write THR	tx_o emits framed bits
TC-URT-004	RX char	Drive rx_i pattern	RBR returns char
TC-URT-005	FIFO enable	FCR.FIFOEN=1, fill TX FIFO	LSR.THRE clears/sets correctly
TC-URT-006	RX trigger IRQ	RXTRIG=4; rx 4 chars	irq_o asserts
TC-URT-007	LSR errors	Inject bad parity	LSR.PE=1; IRQ raised
TC-URT-008	Modem control	Write MCR.RTS	rts_no asserts
TC-URT-009	DMA mode	Set DMA mode	rxrdy_o / txrdy_o behave per spec
TC-URT-010	FIFO parity error	Force flip in FIFO RAM	err_o asserts
TC-URT-011	Loopback	Set MCR.LOOP	Internal loopback OK

B.20 Log Engine

ID	Title	Stimulus	Expected
TC-LOG-001	Single entry transfer	Configure region; LOG_CTRL[0]=100	100 bytes appear at LOG_WRITE_ADDR; LOG_CTRL[0]=0
TC-LOG-002	UART back-pressure	Hold uart_tx_ready_i low	Engine waits; no progress
TC-LOG-003	Round-robin	Set lengths in 3 entries	All transferred fairly
TC-LOG-004	Fetch error	Backing memory returns SLVERR	LOG_FETCH_ERR set; IRQ
TC-LOG-005	Write error	UART write returns SLVERR	LOG_WRITE_ERR set; IRQ
TC-LOG-006	INTR_TEST	Write INTR_TEST	Status bits set
TC-LOG-007	Disabled engine	CTRL=0	No writes occur

B.21 I2C Controller

ID	Title	Stimulus	Expected
TC-I2C-001	Controller write	Push START + ADDR+W + data + STOP via FDATA	i2c_bus model captures it
TC-I2C-002	Controller read	Issue START + ADDR+R + read N	RDATA contains slave bytes
TC-I2C-003	NACK detection	Slave NACKs	irq_nak_o asserts
TC-I2C-004	Clock stretching	Slave model holds SCL	Controller waits
TC-I2C-005	Target mode	Configure as target	Responds to addressed transfer
TC-I2C-006	Multi-controller arbitration	Two controllers contend	Loser backs off
TC-I2C-007	Speed modes	100k / 400k / 1M	Timing reflects setting
TC-I2C-008	FIFO threshold IRQs	Fill RX > threshold	IRQ
TC-I2C-009	Bus monitor mode	Watch traffic	RX shows passive copy

B.22 AVSBus Controller

ID	Title	Stimulus	Expected
TC-AVS-001	Reset values	Read AVS_CFG_0/CFG_1	Match spec defaults
TC-AVS-002	Send commit-write	Push AVS_CMD; observe avs_mdata_o	32-bit frame matches; CRC-3 correct
TC-AVS-003	Receive response	Target sends ACK frame	AVS_READBACK contains data; AVS_SLAVE_STATUS updated
TC-AVS-004	Bad CRC retry	Target returns bad CRC	Auto-retry up to MAX_RETRIES
TC-AVS-005	NACK	Target NACKs	AVS_INTERRUPT raises NACK bit
TC-AVS-006	Resync	RESYNC_INTERVAL ticks	Resync sequence emitted (34 cycles)
TC-AVS-007	FIFO full	Push 9 cmds	FIFO full bit
TC-AVS-008	FIFO empty IRQ	All consumed	Empty bit set
TC-AVS-009	Latest subframe bypass	Read AVS_LATEST_SLAVE_SUBFRAME	Latest valid frame shown
TC-AVS-010	Clock select	Switch CLOCK_SELECT	AVS_clock_o derives from new source
TC-AVS-011	Clock div + duty	Program div=8, duty=128	avs_clock_o period = 8*src; 50%
TC-AVS-012	Stop on idle	STOP_AVS_CLOCK_ON_IDLE=1	clock gates when idle

B.23 GPIO

ID	Title	Stimulus	Expected
TC-GPIO-001	Reset defaults	Read all DATA_CTRL	Match defaults; control bits cleared
TC-GPIO-002	Drive output	Set enable_rx_tx=01, core2pad=1	pad reads 1
TC-GPIO-003	Read input	Drive pad=1	pad2core=1
TC-GPIO-004	Strap capture	Drive pad=1 then deassert reset	strap_value=1; strap_valid=1; captured_strap_o=1
TC-GPIO-005	Edge interrupt	interrupt_type=10; rising edge	irq_o pulses
TC-GPIO-006	Level interrupt	interrupt_type=00; pad held high	irq_o stays high
TC-GPIO-007	LSIO override	lsio_select=1	Pad driven by LSIO data; reg ignored
TC-GPIO-008	Access filter	awprot mismatch	DECERR
TC-GPIO-009	Drive strength reg	Program 6mA → 12mA	CONTROL.drive_strength updated
TC-GPIO-010	All 68 pins	Walk every pin	Each functions

B.24 Telemetry Receiver

ID	Title	Stimulus	Expected
TC-TLM-001	ATB protocol handshake	Send 8 valid beats	atready_o handles flow
TC-TLM-002	Packet assembly	Send 8 bytes	One 64-bit packet built
TC-TLM-003	Decode probe ID + 4 counters	Send valid msg	TELEMETRY_PROBE_ID set; TELEMETRY_COUNTER[0..3] correct
TC-TLM-004	Counter validity	Some counters absent	TELEMETRY_COUNTER_VLDS reflects mask
TC-TLM-005	Buffer threshold IRQ	Threshold=4; insert 4 messages	IRQ asserted
TC-TLM-006	Buffer full	Insert past depth	Status.full=1; new msgs dropped/marked
TC-TLM-007	Flush handshake	Assert afready_i	afvalid_o pulses appropriately
TC-TLM-008	Error detection	Send malformed packet	INTR_STATUS.err set
TC-TLM-009	Multi-source ATID	Mix two atid sources	All accepted (per-ID tracking)

B.25 PVT Wrapper

ID	Title	Stimulus	Expected
TC-PVT-001	Default values	Read	Match reset spec
TC-PVT-002	Force temp value	TB calls force_temperature(85.0)	TEMP_VALUE updated
TC-PVT-003	Threshold IRQ high	Set THRESH_HI=80; force 85	temp_irq_o asserts
TC-PVT-004	Threshold IRQ low	Set THRESH_LO=20; force 10	temp_irq_o asserts
TC-PVT-005	Voltage readback	Force voltage	VOLT_VALUE matches

C. SoC-Level Integration Tests (smc_top)

C.1 Boot sequence (TC-SOC-BOOT-NNN)

ID	Title	Description	Expected
TC-SOC-BOOT-001	Cold boot to OCCP loop	Load production ROM image; release rst_cold_n	After fuse sense and SRAM init, CPU enters OCCP processing loop; POST codes published
TC-SOC-BOOT-002	Strap configuration	Drive specific strap pins (BL0_PLLCLK, BYPASS_SRAM_REPAIR, SRAM_AUTO_ZERO_DISABLE, TEST_EN)	Boot phases match strap-driven branches
TC-SOC-BOOT-003	Primary chiplet boot	Strap configures primary	OCCP commands distributed via I2C/I3C
TC-SOC-BOOT-004	Secondary chiplet boot	Strap configures secondary	Awaits OCCP commands; PLL configured in Phase 1.5
TC-SOC-BOOT-005	Memory repair flow	Provide BIRA data; assert fuse_sense_done	Repair triggered, dft_boot_seq_done received, CPU released
TC-SOC-BOOT-006	Bypass repair via strap	BYPASS_SRAM_REPAIR=1	Repair skipped; SRAM accessed directly

C.2 FLR sequence (TC-SOC-FLR-NNN)

ID	Title	Description	Expected
TC-SOC-FLR-001	Trigger FLR	Pulse cfg_flr_pf_active_i	After PRE counter, isolation asserts; after RESET counter, cool reset asserts; skip_mem_repair_o=1
TC-SOC-FLR-002	Recovery	Allow sequence to finish	SMC re-boots without affecting SoC
TC-SOC-FLR-003	Pin-based isolation	Assert isolate_req_pin_i with PINEN	Output isolate_req_o asserts

C.3 Interrupt aggregation (TC-SOC-IRQ-NNN)

ID	Title	Description	Expected
TC-SOC-IRQ-001	Mailbox path	Push to inbound mailbox 5	PLIC #(288+5) fires; sync_irq_o high
TC-SOC-IRQ-002	UART path	Generate UART RX	PLIC #275 fires
TC-SOC-IRQ-003	I2C path	I2C controller raises IRQ	PLIC #279..281 fire
TC-SOC-IRQ-004	AVSBus path	AVSBus IRQ	PLIC #278 fires
TC-SOC-IRQ-005	Telemetry path	Threshold reached	PLIC #264..266 fires
TC-SOC-IRQ-006	Temp sensor	force_temperature triggers	PLIC #283 fires
TC-SOC-IRQ-007	NDM reset	Write NDM_RESET_REQ	PLIC #267 fires
TC-SOC-IRQ-008	All 326 active sources	Walk-test (regression)	All deliver
TC-SOC-IRQ-009	BEU as NMI	Cause BEU error	NMI delivered to corresponding hart, bypassing PLIC

C.4 DMA + fabric scenarios (TC-SOC-DMA-NNN)

ID	Title	Description	Expected
TC-SOC-DMA-001	SRAM→SRAM copy	DMA copy 64KB inside scratchpad	Bit-perfect; IRQ fires
TC-SOC-DMA-002	SRAM→external	DMA src=SRAM, dst=output_axi region	Forwarded via output port
TC-SOC-DMA-003	Filter blocks	Configure outbound filter to block dst	DECERR; STATUS.error=1
TC-SOC-DMA-004	Alias remap path	DMA via alias-remapped region	Address translation correct
TC-SOC-DMA-005	Concurrent DMA + CPU	CPU writes mailbox while DMA runs	Both complete; no corruption

C.5 OCTS multi-chiplet (TC-SOC-OCTS-NNN)

ID	Title	Description	Expected
TC-SOC-OCTS-001	1 PRIMARY + 3 SECONDARY synchronization	Wire pulses between 4 instances	All counts within tolerance after 1ms
TC-SOC-OCTS-002	Drift recovery	Simulate dropped credit pulse	CREDIT_EXPIRED rises; recovery on next pulse
TC-SOC-OCTS-003	Timestamp service	CPU calls get_timestamp via TLM helper	Stable monotonic 64-bit value

C.6 eFuse + Security (TC-SOC-SEC-NNN)

ID	Title	Description	Expected
TC-SOC-SEC-001	Lifecycle PROD restricts JTAG	Set lc_state=PROD; JTAG read regs	Blocked except chiplet ID
TC-SOC-SEC-002	Token-based unlock	Provide RMA SOP token	MMR access granted
TC-SOC-SEC-003	Filter NS-blocked peripheral	Mark UART region NS-blocked; non-secure CPU access	DECERR returned to CPU
TC-SOC-SEC-004	Security disable	Toggle security_disable_i during run	Shadow regs become directly accessible

C.7 Logging (TC-SOC-LOG-NNN)

ID	Title	Description	Expected
TC-SOC-LOG-001	Log → UART end-to-end	Log engine fetches from SRAM, writes to UART	UART receives stream of bytes; visible at attach_stdout
TC-SOC-LOG-002	Multiple log producers	All 4 log engines active	UART order shows fair interleaving

C.8 Telemetry (TC-SOC-TLM-NNN)

ID	Title	Description	Expected
TC-SOC-TLM-001	ATB stream from TB	Inject 100 messages	Telemetry CPU-readable; no losses
TC-SOC-TLM-002	Threshold IRQ → CPU	Threshold reached	CPU receives PLIC IRQ #264
TC-SOC-TLM-003	Three telemetry receivers concurrent	Three streams	Each independently functional

C.9 Stress / Random (TC-SOC-STR-NNN)

ID	Title	Description	Expected
TC-SOC-STR-001	Random reg walk	Random R/W to all regs (except destructive)	No crash; reset values restored after reset
TC-SOC-STR-002	Random IRQ injection	Random sources for 1 hour sim	No deadlocks; no missed IRQ in claim/complete sequencing
TC-SOC-STR-003	DMA + Zeroer + Log Engine concurrent	All masters active	No fabric deadlock; throughput recorded

D. Functional Coverage Plan

For each IP, the following functional coverpoints are tracked:

- **Reset:** every reset source x every reset domain.
- **Register access:** every register, every legal R/W combination, plus illegal accesses (non-aligned, wrong width).
- **Interrupts:** every IRQ source asserted at least once; cleared via every documented mechanism.
- **Modes:** every documented mode entered (e.g. PRIMARY/SECONDARY OCTS, controller/target/monitor I2C, FIFO on/off UART, FLR vs cold reset).
- **Errors:** every error path executed (DECERR, SLVERR, CRC error, FIFO overflow/underflow, parity error).
- **Cross-IP:** every interrupt source delivered through PLIC; every filter configuration combined with at least one transaction.

A `coverage.csv` is emitted at end of each test; `tools/merge_coverage.py` aggregates across runs.

E. Test Deliverables

File / Directory	Contents
<code>tests/unit/<ip>/</code>	One GoogleTest binary per IP
<code>tests/soc/</code>	smc_top integration tests
<code>tests/stim/</code>	Stimulus payloads (firmware images, ATB streams, AVSBus targets, I2C slave models)
<code>tests/cov/</code>	Coverage scripts + reports
<code>tests/CMakeLists.txt</code>	Build description
<code>Jenkinsfile</code> / <code>.github/workflows/ci.yml</code>	Tier-0/1/2 schedules
<code>docs/test_results_template.md</code>	Per-release report template

F. Open Items / Out of Scope

- **Cycle-accurate (AT)** modeling — out of scope for the LT library; this plan only verifies LT functional behavior.
- **I3C controllers (§6.10)** — RESERVED in spec; not modeled; placeholder for future addition.
- **Real silicon timing closure** — handled at RTL stage; not exercised here.
- **Formal verification** — not in scope; complementary effort.

This plan, together with `01_SMC_Architecture.md` and `02_SMC_IP_LowLevel_Design.md`, fully specifies the modeling and verification of the SMC subsystem in Accellera SystemC/TLM-2.0 (Loosely Timed).